



WAMS Project

Microservice-Based Application for Managing Medical Institution Data and Interacting with AI Models

**BOUSSENSLA Hadil
MAHIDDINE Islem Amine
TALAHARI Yassine
YOUSFI Nacer
ZORGANI Alaaeddine**

Advanced Web and Microservices - University of Boumerdes -
Department of Computer Science - Match 2026

1. Introduction

In modern medical AI systems, the gap between a high-performing model and a clinically usable tool is often bridged by the underlying software architecture. Rather than exposing models as standalone scripts, we designed MediSync — a microservices-based medical records platform that enables real-time inference, modular extensibility, and secure access for healthcare practitioners.

1.1. Project context

We developed MediSync as a web-based platform for managing medical records, designed for healthcare professionals across multiple specialties. The platform allows doctors to manage their patients, schedule appointments, upload medical files, and access AI-powered diagnostic tools tailored to their specialty.

The application domain sits at the intersection of healthcare and modern software engineering. We built the platform around a microservices architecture, where each concern — authentication, patient management, AI inference, and the user interface — is handled by an independent, deployable service. This design allows the platform to scale across multiple hospitals or clinics, enabling cross-institution collaboration and centralized patient data management.

The deep learning component focuses on brain tumor analysis from MRI scans, serving as the first AI model integrated into the platform through a model registry that dynamically exposes relevant tools based on the doctor's specialty.

1.2. Problem Statement

Patient records are scattered across incompatible systems, cross-institution collaboration is difficult, and AI diagnostic tools remain isolated from clinical workflows. Existing platforms also treat all doctors the same, ignoring specialty-specific needs.

Our platform addresses these problems through a microservices architecture, a specialty-aware access system, and a model registry that dynamically surfaces AI tools based on the doctor's profile.

1.3. Project Objectives

- Provide doctors with a personalized space based on their specialty

- Manage patients, appointments, and medical files securely
- Enable cross-institution sharing of medical records
- Integrate AI diagnostic models through a specialty-aware model registry
- Build a scalable and extensible microservices architecture.

1.4. Why This Project Is Interesting

MediSync sits at the intersection of two fast-moving fields — modern distributed systems and medical AI. By combining a production-ready microservices backend with real deep learning models trained on the BRISC2025 benchmark, we built something that goes beyond a typical academic exercise. The platform is realistic enough to deploy across multiple machines, extensible enough to plug in new AI models for any specialty, and concrete enough to demonstrate with actual MRI-based tumor classification and segmentation results.

1.5. General Justification

In software engineering, a microservices architecture decomposes an application into a set of small, independently deployable services, each responsible for a single well-defined function. Unlike monolithic applications where all logic is bundled together, each service owns its data, exposes a well-defined API, and can be developed, deployed, and scaled independently.

We chose this approach because MediSync naturally decomposes into distinct concerns — identity management, patient data, AI inference, and the user interface — each with different scaling needs, different technology requirements, and different rates of change.

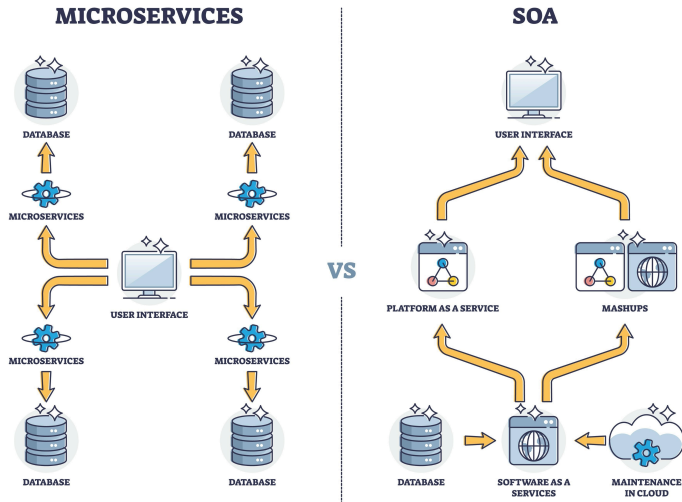


Figure 1: architecture diagram.

1.6. Separation of Responsibilities

1.6.1. Auth Service

The Auth Service manages everything related to identity — user accounts, doctor profiles, medical specialties, and group-based permissions. It is the only service that writes to the authentication database, and it expresses authorization decisions as claims embedded in JWT tokens rather than exposing its database to other services.

1.6.2. Patient Service

The Patient Service is responsible for all clinical data — patient records, appointments, and medical file uploads. It never queries the Auth database directly. Instead, it reconstructs the user's identity and permissions from the JWT token alone, achieving true statelessness.

1.6.3. AI Service

The AI Service handles inference exclusively. It receives medical images, routes them to the appropriate model based on the doctor's specialty, and returns predictions. It maintains no persistent data of its own.

1.6.4. Frontend Service

The Frontend Service handles rendering and user interaction. It communicates with backend services only through the Traefik gateway and never bypasses the API layer.

1.7. Scalability and Flexibility

Because each service is independently deployable, we can scale only the parts that need it. If AI inference becomes a bottleneck, we can run multiple instances of the AI Service behind Traefik without touching the Auth or Patient services. If a new hospital joins the network, we can spin up additional Patient Service instances on a separate machine and register them with Consul — the rest of the system adapts automatically.

The same logic applies to future extensions. Adding a new AI model for a different specialty means deploying a new service and registering it in the model registry, without modifying existing services. This makes MediSync extensible by design, not by accident.

1.8. Independent Development and Deployment

Each service has its own codebase, its own dependencies, and its own Dockerfile. We can build, test, and deploy the Patient Service without touching the Auth Service, and update the AI Service without redeploying the frontend.

This also meant that during development, each team member could work on a different service simultaneously without stepping on each other. The test suite for each service runs in isolation, and the full end-to-end test only requires the services to be running — not their internals to be coupled in any way.

1.9. Technical Fit

The microservices architecture maps directly onto our technical requirements. Authentication is handled by a dedicated service that issues JWT tokens, allowing every other service to verify identity without inter-service calls. Service discovery and routing are managed by Consul and

Traefik, meaning services can register and deregister dynamically without hardcoded addresses. The multi-machine setup distributes services across physical machines connected over LAN, with Consul's gossip protocol keeping the registry consistent across nodes. The AI Service benefits from asynchronous task handling via FastAPI, allowing inference requests to be processed without blocking the event loop. Each of these requirements found a natural home in the architecture rather than being forced into it.

2. Microservice architecture

2.1. Overall architecture view

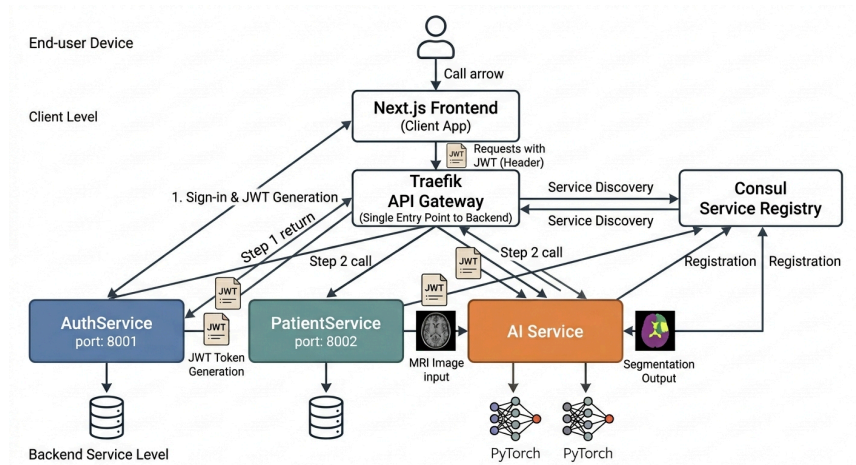


Figure 2: Global Application Architecture

The platform is structured as a set of four loosely coupled microservices, each with a distinct responsibility, communicating through a shared API gateway rather than directly with one another. Traefik serves as this gateway, acting as the single external entry point and routing incoming requests to the appropriate service based on path prefixes and the service catalog maintained by Consul. The AuthService handles all identity concerns, issuing JWT tokens that carry authorization claims consumed by every other service. The Patient Service manages clinical data including patient records, appointments, and medical files, relying entirely on token claims for authorization rather than maintaining any user state of its own. The AI Service provides stateless ML inference for medical imaging, receiving MRI uploads, running classification and segmentation models, and returning structured predictions without

persisting any patient data. The Frontend, built with Next.js, sits at the user-facing edge and communicates exclusively through Traefik, adapting its interface dynamically based on the authenticated user's role and permissions. Each service owns its own database where persistence is needed, and none share data stores, ensuring that service boundaries are enforced at every layer of the stack. This architecture was chosen to demonstrate how independent deployability, stateless authentication, and dynamic service discovery can be applied cohesively within a single healthcare platform.

2.2. Service decomposition

2.2.1. Auth Service

The AuthService is a Django REST Framework application handling user management and authentication on port 8001, backed by its own PostgreSQL database. Upon login, it issues JWT access tokens embedding group memberships, permissions, superuser status, and doctor specialty, allowing downstream services to authorize requests without querying Auth again. Two groups are defined: Admin, with full management capabilities, and Doctor, with limited administrative rights but broad patient-data permissions. Doctor profiles extend the base user model with a specialty field, and group assignments are automated through Django signals post-migration. The service registers with Consul at startup, exposing a health check endpoint that Traefik uses for routing.

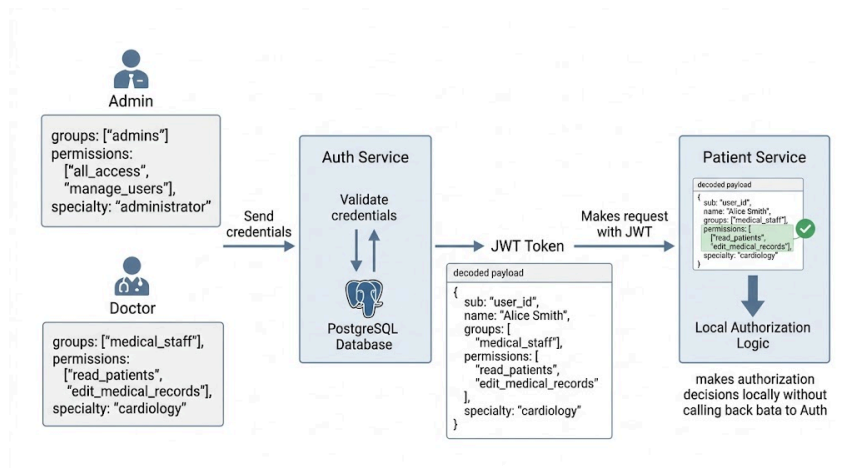


Figure 3: How the Auth Service Works

2.2.2. Patient Service

The Patient Service runs on port 8002 and manages patient records, appointments, and medical file metadata in its own PostgreSQL database. It never queries the Auth database directly; instead, a custom `StatelessJWTAuthentication` class reconstructs a lightweight `ServiceUser` from token claims on each request. Patient records store demographic and clinical data, support multi-doctor assignment via a JSON array of user identifiers, and include duplicate detection by phone number. Appointments track status across scheduled, completed, cancelled, and expired states, with automatic expiry on retrieval. Medical files are stored on the filesystem with metadata in the database, and cascading deletes on patient removal clean up all associated records. The service registers with Consul for Traefik-based routing.

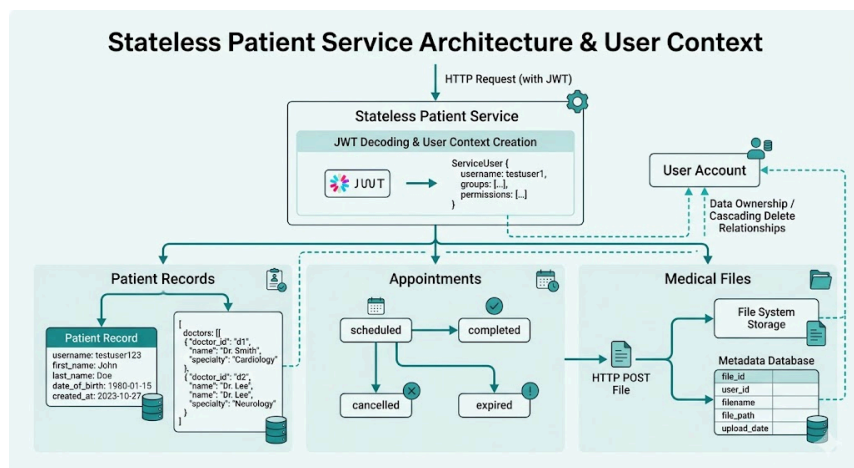


Figure 4: How the Patient Service Works

2.2.3. Frontend Service

The Frontend Service is a Next.js application serving as the UI layer, communicating exclusively through the Traefik gateway rather than directly with any backend service. Public pages use server-side rendering while authenticated dashboard views switch to client-side rendering, with the JWT stored in HTTP-only cookies. The interface adapts to the user's role: admins see user management controls while doctors see their patient panel and appointment scheduler. Patient, appointment, and medical file management are all handled through form-based interfaces with real-time validation, paginated listings, and confirmation dialogs for destructive operations. Token

refresh occurs transparently, and toast notifications and loading states provide appropriate feedback throughout.

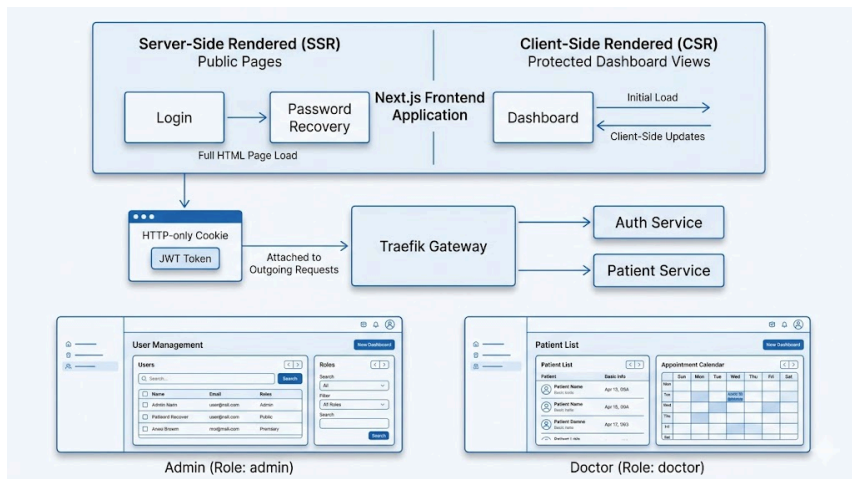


Figure 5: How the Frontend Works

2.2.4. AI Service

The AI Service is a stateless FastAPI microservice providing ML inference for medical images, running without a persistent database to allow horizontal scaling. Incoming requests pass through Traefik, which forwards JWT claims to the service; the service uses the doctor's specialty to determine which models they may access. Two PyTorch models are loaded at startup for the brain tumor use case: a classifier outputting tumor-type probabilities and a segmentation model producing pixel-wise masks. Uploads are handled asynchronously, with inference running in a background task and results returned via polling or WebSocket. The preprocessing pipeline handles resampling, intensity normalization, and DICOM metadata stripping before inference. Results include the predicted class, confidence scores, and a base64-encoded segmentation mask ready for frontend overlay. The service registers with Consul, and Traefik applies appropriate rate limits and timeouts given the latency of inference workloads.

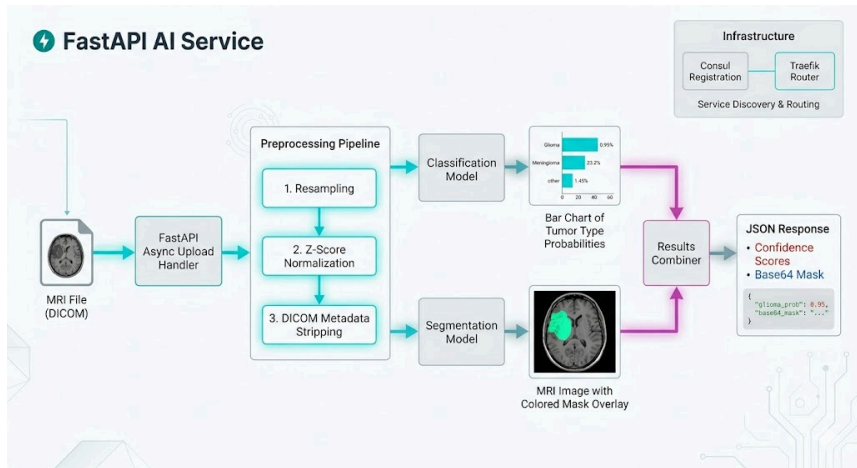


Figure 6: How the AI Service Works

2.3. REST APIs and Data Management

The backend exposes RESTful APIs that allow communication between the frontend and backend services.

The main API categories include:

2.3.1. Patient APIs

- Create, update, retrieve, and delete patient records.
- Search and filter patients.
- Access patient medical history.

2.3.2. Appointment APIs

- Schedule appointments.
- Update appointment status.
- Track expired or completed consultations.

2.3.3. Medical File APIs

- Upload clinical documents and imaging reports.
- Retrieve patient-related files.
- Delete obsolete records.

2.3.4. AI APIs

- Upload MRI scans.
- Request classification and segmentation inference.

- Retrieve prediction results and confidence scores.

PostgreSQL is used as the primary database system for persistent services. Each microservice maintains its own isolated database, reinforcing service independence and reducing coupling between components.

2.4. Service Communication

Services in this platform communicate exclusively over HTTP, with the Traefik gateway acting as the single entry point for all external requests. Authentication is handled by the AuthService, which issues signed JWT tokens upon login. These tokens carry embedded claims including group memberships, permissions, and doctor specialty, allowing the Patient Service and AI Service to authorize requests locally without querying the AuthService on each call. This stateless design eliminates inter-service dependencies at request time: each service receives a token, decodes it independently, and makes authorization decisions based solely on its claims. The Frontend communicates with backend services entirely through Traefik, never holding direct references to individual service addresses, which keeps the client decoupled from the internal topology.

2.5. Infrastructure and Tooling

The platform is containerized using Docker, with each microservice running in its own isolated container alongside its dependencies. Service discovery is handled by Consul, with each service registering its hostname, port, and health check endpoint at startup. Traefik reads from Consul's service catalog to dynamically build its routing table, eliminating the need for hardcoded addresses or manual proxy configuration. This setup allows services to be added, removed, or restarted without modifying the gateway configuration. The AI Service leverages FastAPI's asynchronous request handling to manage the latency of ML inference workloads, offloading inference to background tasks and returning results through polling or WebSocket connections rather than blocking the request cycle.

2.6. Deployment Model

Each microservice is deployed as an independent unit with its own codebase, dependencies, and where applicable, its own PostgreSQL database instance. This isolation means that a change or failure in one service does not directly affect the others, and each can be scaled horizontally by running additional instances behind Traefik. The AuthService and Patient Service each own their

data entirely, with no shared database access between them, enforcing strict service boundaries at the persistence layer. The AI Service is intentionally stateless, holding no persistent data, which makes it straightforward to scale under concurrent inference load. This deployment strategy aligns with the academic scope of the project by demonstrating how independent deployability and separation of concerns can be achieved even within a relatively small system.

3. Frontend Design Details

3.1. Main screens and flows

The MediSync user interface is designed to provide a seamless and intuitive experience for healthcare professionals. Developed with a user-centric approach, it allows rapid navigation between patient records and diagnostic support tools.

3.1.1. Authentication Interface

Security is at the heart of MediSync. The login interface ensures restricted access to sensitive patient data via a robust authentication system.

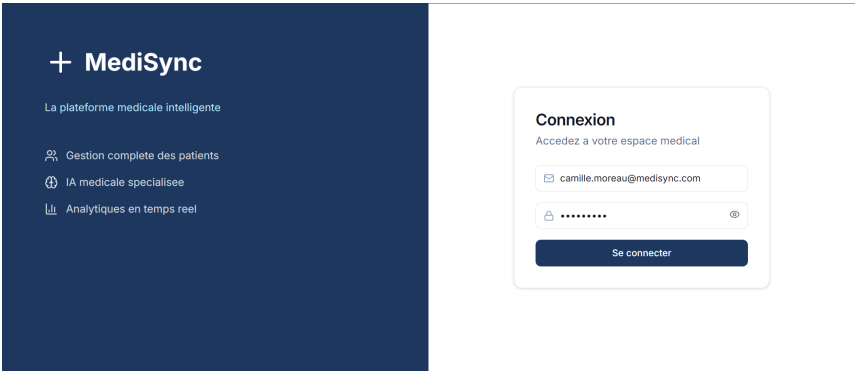


Figure 7: MediSync Secure Login Page

3.1.2. Dashboard

The dashboard serves as the central hub for the practitioner. It offers a synthetic view of clinical activity:

- **Key Performance Indicators (KPIs):** Total patient count, new patients for the month, and AI analysis volume.
- **Analytics:** Graphical visualization of consultation trends over time.

- **Demographic Breakdown:** A circular chart showing the gender distribution of the patient base.

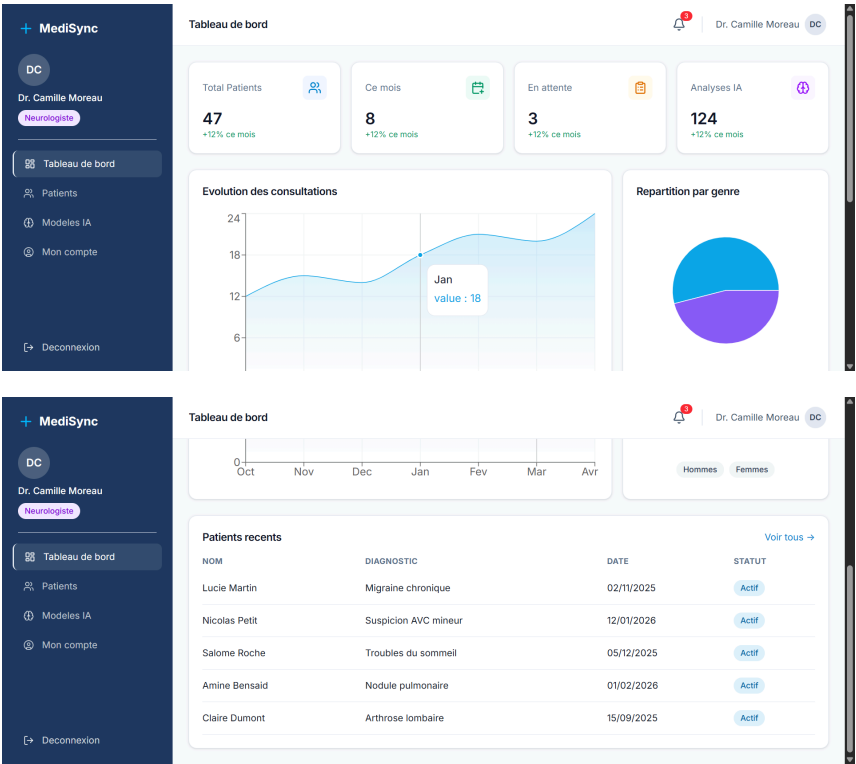


Figure 8: Detailed views of the practitioner’s dashboard: analytics and statistics.

3.1.3. Patient Management

This section enables rigorous follow-up of medical records. The interactive table displays vital information (contact details, latest diagnosis) and allows for real-time profile creation or modification.

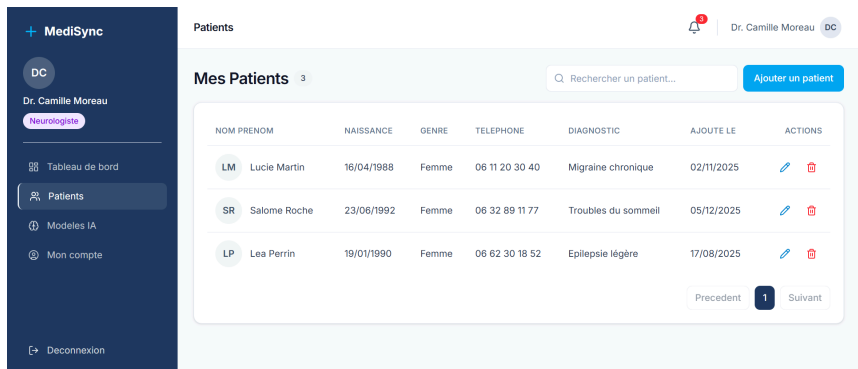


Figure 9: Patient management interface and clinical list

3.1.4. AI Module: NeuroAI

The core innovation lies in the integration of specialized AI models. The module allows practitioners to input clinical symptoms (e.g., hemiplegia, aphasia) to obtain an instantaneous preliminary analysis, facilitating clinical decision-making.

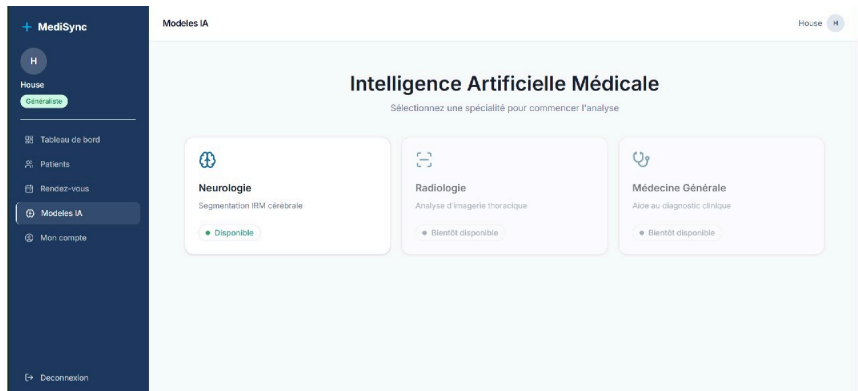


Figure 10: AI-assisted diagnostic tool (NeuroAI)

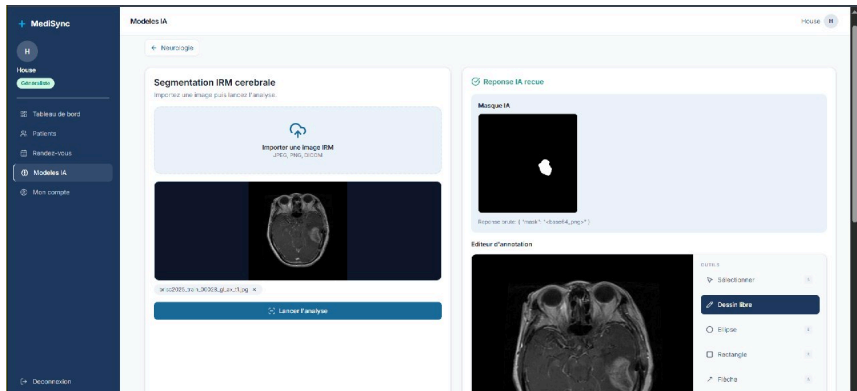


Figure 11: AI-assisted diagnostic tool (NeuroAI)

3.1.5. Account Settings

The practitioner can manage personal information and account security (such as password updates) through a dedicated profile interface.

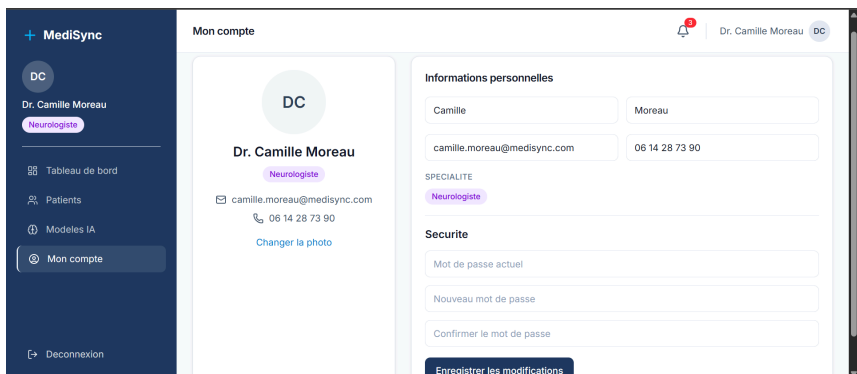


Figure 12: User profile and security management

3.2. Conclusion

This project set out to design and implement a modular healthcare platform integrating a microservices backend with machine learning inference capabilities, and the final system delivers on that scope across four independently deployable services covering authentication, patient management, clinical AI, and a role-aware frontend. The segmentation pipeline achieved a peak Dice score of 0.8696, surpassing the published BRISC 2025 baseline, while the classification models reached accuracies above 0.99

on the best configurations. Beyond raw performance, the architectural choices made throughout the project carry practical value: stateless JWT-based authorization eliminates inter-service coupling at request time, Consul and Traefik together enable dynamic routing without hardcoded addresses, and strict database-per-service boundaries ensure that each component can evolve or scale without affecting the others. The result is a system that is not only functional but maintainable and extensible by design. Looking ahead, several directions could strengthen the platform further: introducing an asynchronous message broker such as RabbitMQ or Kafka for event-driven coordination between services, expanding the AI Service to support additional medical specialties and model types, adding centralized logging and distributed tracing for production-grade observability, and automating deployment through a container orchestration layer such as Kubernetes to move beyond the current single-host setup.